

LASTING_cluster_w_masterinput_documentation

Fred Cudhea

09 June, 2025

R Markdown

Documentation for R script: LASTING_cluser_w_masterinput.r which runs the LASTING model on the cluster. Unless you are making substantial deviations to the methodology, this should be the only script you need to edit to run your analysis. I'm documenting the code as is, and so am keeping preserved but obsolete code (under the philosophy that it's better to keep in case it becomes useful later, rather than lose it). In general, you do not need to worry about code that is commented out (though you may find it useful, especially the ones for debugging). If there is commented out code that might potentially be useful, it will be noted, but none of the commented out code is necessary for the model to run. You can find all the scripts called by this script, and the scripts called by those scripts, (the overall nesting structure of the code) spelled out in the Code annotation read me file (alongside a short summary of what each script does).

Recall the general idea: Calculate impact of three of the pillars by multiplying impact by change in intake. Then, calculate health impact using CRA. In all pillars, uncertainty of inputs is propagated into outputs via Monte Carlo simulations. In other words, we are sampling 1000 potential values for the inputs from a distribution, and calculating the impacts 1000 times. Therefore, we are going to have a distribution of inputs and outputs. These documents will refer to these as sims, simulations, draws, etc... interchangeably.

First, we clean the environment, load necessary R packages, set up pathways for code and input files.

```
## The definitive r script for running PIF Analysis. Hopefully, using this will cut down on time spent
## changing things around for various sensitivity / alternative analysis.

# Authors: Fred Cudhea & Brooke Bell
# Last Updated: 12-11-24

##### SET UP #####

rm(list=ls(all=TRUE)) # remove all data from memory

library(plyr)
library(data.table)
# library(dplyr)
# library(tidyverse)
# for whatever reason, Brooke can't install tidyverse on the cluster
# load tidyverse packages individually
library(ggplot2)
library(dplyr)
library(tidyr)
library(readr)
library(purrr)
library(tibble)
library(stringr)
library(forcats)
library(foreach)
library(doParallel)

# File Location
file_location = "/cluster/tufts/lasting/shared/LASTING/in/FINAL"
# file_location = "/Users/bmb73/Documents/GitHub/LASTING/in/FINAL"
setwd(file_location)

# Code Location
code_location = "/cluster/tufts/lasting/shared/LASTING/Code/cluster_code"
# code_location = "/Users/bmb73/Documents/GitHub/LASTING/Code/cluster_code"

# Set the seed
set.seed(3172023)
```

We have two “version.date” objects here because the many of health files used only for the CRA were created earlier than the main MEGA file used for all 4 pillars. In lieu of re-writing the code so that they all share the same version dates, we allow for two different version dates here.

```
# Change this when date of data inputs is updated
version.date <- "2025-03-10"
health_files_version.date <- "2025-03-10"
```

Start clock so can get an idea of how long the code takes to run

```
start_time <- Sys.time()
```

year.vec.string allows us to include the specified years of interest as part of the output names. Note that this is strictly for naming output files. The code itself only uses input files that correspond to a particular time point. It does not calculate estimates for multiple time points nor should you be feeding it input files with multiple time points. If you are doing analyses for multiple time points, the easiest way to do it is to have input files for each time point of interest, and then the rerunning the code, making sure to change “year.vec” here so that the output file names differ by year.

```
year.vec <- 2015

if(length(year.vec) > 1) {
  year.vec.string<-paste(year.vec[1], "-", year.vec[length(year.vec)], sep="")
}else{
  year.vec.string <- as.character(year.vec)}

```

Set seed for reproducibility. You can choose whatever number you like for the seed, but make sure you're always using the same seed value when rerunning the code. You may have noticed the seed was `seed` earlier in the code. This is because someone (probably me) forgot the seed was already set and added either this line or the one above. In any case, this line, which comes later, will overwrite the previous one, so the seed is one for this script.

```
set.seed(1)
```

Set number of simulations to 1000

```
# Number of simulations
nsim1 <- 1000
n.sims <- nsim1

```

Specify the subgroups of interest. The names used for the subgroups should match what is used in the mega input file.

For the purposes of these documents let's formalize some of the terms to reduce confusion. "Subgroup" will denote the 48 unique age/sex/race combinations that we divide the population into. The "strata" are the variables used to divide up a data set. For example, the 48 subgroups are a result of dividing up the population using three strata: age, sex, race. Age/sex/race is one example of a "strata combination".

```
# Covariates vector
covar.vec <- c("Age", "Sex", "Race") ##Don't mess with the order age, sex, race
covar.vec.string <- paste(covar.vec, sep="", collapse="")
n.covar <- length(covar.vec)

```

Obsolete. Not used anymore (previously used to denote specific input files)

```
# this is for exposure data
incidence.version.date <- "2015_v7_25_18"

```

Identify all dietary patterns of interest here. Names should match "diet_pattern" column in mega input file (except not really because mega input file uses "Vegetarian" instead of "Veg" and we rename it later in the code).

```
# Vector of dietary patterns
diet.vec <- c("US",
             "Veg",
             "Med",
             "Vegan",
             "DGAplus"
            )

```

Identify all environmental, economic, social pillar outcomes. Names should match what exists in mega file. "all.outcomes.vec" looks to be obsolete.

```
##### COST/ENV-SPECIFIC INPUTS #####
```

```
# Enviro and economic outcomes vector
```

```
econ.outcomes.vec <- c(
  "CED",
  "GHG",
  "WATER",
  "BLUEWATER",
  "Food_price",
  "FL"
)
```

```
all.outcomes.vec <- c(
  "CED",
  "GHG",
  "WATER",
  "BLUEWATER",
  "Food_price",
  "FL"
)
```

Create vectors to store outcomes

```
# Create some more vectors
```

```
n.econ.outcomes <- length(econ.outcomes.vec)
econ.outcomes.vec.mn <- paste("Mean", econ.outcomes.vec, sep="_")
econ.outcomes.vec.se <- paste("SE", econ.outcomes.vec, sep="_")
econ.current.inedible.p.mn <- paste(econ.outcomes.vec, "_inedible_p", sep="")
econ.current.inedible.p.se <- paste(econ.outcomes.vec, "_inedible_p_se", sep="")
econ.counterfactual.inedible.p.mn <- paste("CF_",econ.outcomes.vec, "_inedible_p", sep="")
econ.counterfactual.inedible.p.se <- paste("CF_",econ.outcomes.vec, "_inedible_p_se", sep="")
econ.current.foodwaste.p.mn <- paste(econ.outcomes.vec, "_foodwaste_p", sep="")
econ.current.foodwaste.p.se <- paste(econ.outcomes.vec, "_foodwaste_p_se", sep="")
econ.counterfactual.foodwaste.p.mn <- paste("CF_",econ.outcomes.vec, "_foodwaste_p", sep="")
econ.counterfactual.foodwaste.p.se <- paste("CF_",econ.outcomes.vec, "_foodwaste_p_se", sep="")
num.econ.outcomes <- length(econ.outcomes.vec.mn)

econ.outcomes.substitution.vec.mn <- paste("Mean_substitution", econ.outcomes.vec, sep="_")
econ.outcomes.substitution.vec.se <- paste("SE_substitution", econ.outcomes.vec, sep="_")
```

List dietary vectors of interest here. Certain dietary factors are only of interest for environment/labor, others only for cost. But all dietary factors of interest in those three pillars should be listed here. Names should match what is used in mega input file.

```
# Dietary factors vector (for cost/env)
rfvec_costenv <- c("dairy_tot", #cost
                  "dairy_cow", #enviro
                  "dairy_soy", #enviro

                  "gr_whole",
                  "gr_refined",

                  "pf_egg",
                  "pf_pm", #cost
                  "pf_redm", #cost
                  "pf_redm_tot", #enviro
                  "pf_poultry", #cost
                  "pf_poultry_tot", #enviro
                  "pf_seafood",
                  "pf_ns",
                  "leg_tot",

                  "veg_dg",
                  "veg_ro",
                  "veg_oth",
                  "veg_sta",

                  "fruit_exc_juice",
                  "fruit_juice",

                  "added_sugar",
                  "oil",
                  "sat_fat")
```

Code to create folders to store sensitivity analyses. Commented out because it only needs to be run once to create the folders. You may want to uncomment and adapt as necessary if you are considering doing sensitivity analyses.

```
# sensitivity analyses
# sensitivity.vector<-c(.1, .2)
##create folders for sensitivity
# for(i in 1:length(diet.vec))
# {
#   for(j in 1:length(sensitivity.vector))
#   {
#     sensitivity_folder_path<-file.path("/cluster/tufts/lasting/shared/model_development/data/outputs/CRA",
#                                         diet.vec[i], paste("sensitivity_", sensitivity.vector[j], sep=""))
#     dir.create(sensitivity_folder_path)
#     Sys.chmod(sensitivity_folder_path, "770")
#     joint_sensitivity_folder_path<-file.path("/cluster/tufts/lasting/shared/model_development/data/outputs/CRA",
#                                              diet.vec[i], "joint" ,paste("sensitivity_", sensitivity.vector[j], sep=""))
#     dir.create(joint_sensitivity_folder_path)
#     Sys.chmod(joint_sensitivity_folder_path, "770")
#   }
# }
```

Import mega input file and rename some variables, subset for cost/env/FL analysis

```
# IMPORT MASTER DATA FILE

# Import exposure filep
master_input <- read.csv(file=paste("mega_costenv_structure_", version.date, "_FINAL.csv", sep=""), header=T)

# fix veg name
master_input$diet_pattern[master_input$diet_pattern=="Vegetarian"] <- "Veg"

# fix two var names
master_input <- master_input %>%
  mutate(population = Population_size,
         subgroup = subgroup_id)

# subset to only include dietary factors for cost/env analysis
master_input_costenv <- subset(master_input, Foodgroup %in% rfvec_costenv)
```

Create lists to store input and locations for output (by diet pattern and grocery/non-grocery). Each element of a list corresponds to a particular dietary pattern and “datatype” (grocery vs non-grocery).

```
# Create some more vectors
costs_inputs.vec <- list()
costs_output_loc.vec <- list()

# debug
# i=1
for (i in 1:length(diet.vec)) {

  costs_inputs.vec[[paste0(diet.vec[[i]], "_fah")]] <- subset(master_input_costenv, diet_pattern==diet.vec[[i]] &
  costs_inputs.vec[[paste0(diet.vec[[i]], "_fafh")]] <- subset(master_input_costenv, diet_pattern==diet.vec[[i]]

  # Output Locations vector
  costs_output_loc.vec[[paste0(diet.vec[[i]], "_fah")]] <- paste0(diet.vec[[i]], "_diet_Gro")
  costs_output_loc.vec[[paste0(diet.vec[[i]], "_fafh")]] <- paste0(diet.vec[[i]], "_diet_Oth")

}
```

Read in subgroup specific population numbers.

```
# Population
pop <- costs_inputs.vec[[1]]

pop <- pop %>%
  mutate(Sex = as.character(Sex)) %>%
  filter(datatype == "Grocery") %>%
  select(subgroup, Sex, Age_label, Race_label, population) %>%
  distinct() %>%
  mutate(population.se = 0) %>%
  filter(!is.na(subgroup))
```

Next, simulate distribution for total population numbers (in our cases, since population standard errors are set to 0, we're just repeating the same pop count for each simulation).

```
# Pre-simulate mortality/incidence numbers, to be used in CRA simulations, as well for calculating reverse-engine
# Also, pre-simulate total population numbers
source(paste0(code_location, "/presimulate.pop.draws_cluster.r"))
```

Run the cost/env/FL simulations via a for loop (for each pair of dietary pattern and grocery/non-grocery combo)

```
##### RUN COST/ENVIRONMENT SIMULATIONS #####  
  
setwd("/cluster/tufts/lasting/shared/LASTING/Code/cluster_code")  
for(k in 1:length(costs_inputs.vec)){  
  
  setwd("/cluster/tufts/lasting/shared/LASTING/Code/cluster_code")  
  print(k)  
  source("Cost_change_source_cluster.r")  
}
```

Create vectors used to name output files and used to define variable names in summary output files.

```

# Dietary factors vector
# diet.vec<-c("US", "Veg", "Med")

output.type.vec.sims<-c("costenv.sims.granular", "substitution.costenv.sims.granular", "combined.costenvs.sims.gr
    "delta.costenv.sims.granular",
    #"costenv.sims.granular.consumed", "substitution.costenv.sims.granular.consumed",
    "combined.costenvs.sims.granular.consumed",
    #"costenv.sims.granular.unconsumed", "substitution.costenv.sims.granular.unconsumed",
    "combined.costenvs.sims.granular.unconsumed",
    "current.intake.costenv.sims.granular", "CF.costenv.intake.sims.granular", "current.intake.cos
    "CF.intake.costenv.sims.granular.consumed", "current.intake.costenv.sims.granular.unconsumed",
    "costenv.sims.granular.inedible", "costenv.sims.granular.wasted",
    "current.intake.costenv.sims.granular.inedible", "CF.intake.costenv.sims.granular.inedible",
    "current.intake.costenv.sims.granular.wasted", "CF.intake.costenv.sims.granular.wasted"
    )

output.type.vec.summary<-c("costenv.summary.granular", "substitution.costenv.summary.granular", "combined.costenv
    "delta.costenv.summary.granular",
    #"costenv.summary.granular.consumed", "substitution.costenv.summary.granular.consumed",
    "combined.costenvs.summary.granular.consumed",
    #"costenv.summary.granular.unconsumed", "substitution.costenv.summary.granular.unconsumed",
    "combined.costenvs.summary.granular.unconsumed",
    "current.intake.costenv.summary.granular", "CF.costenv.intake.summary.granular", "current.inta
    "CF.intake.costenv.summary.granular.consumed", "current.intake.costenv.summary.granular.uncons
    "costenv.summary.granular.inedible", "costenv.summary.granular.wasted",
    "current.intake.costenv.summary.granular.inedible", "CF.intake.costenv.summary.granular.inedib
    "current.intake.costenv.summary.granular.wasted", "CF.intake.costenv.summary.granular.wasted"
    )

output.type.vec.sims.bystrata<-c("costenv.sims", "substitution.costenv.sims", "combined.costenvs.sims",
    "delta.costenv.sims",
    #"costenv.sims.consumed", "substitution.costenv.sims.consumed",
    "combined.costenvs.sims.consumed",
    #"costenv.sims.unconsumed", "substitution.costenv.sims.unconsumed",
    "combined.costenvs.sims.unconsumed",
    "current.intake.costenv.sims", "CF.costenv.intake.sims", "current.intake.costenv.sims.con
    "CF.intake.costenv.sims.consumed", "current.intake.costenv.sims.unconsumed", "CF.intake.c
    "inedible.costenv.sims", "wasted.costenv.sims",
    "current.intake.costenv.sims.inedible", "CF.intake.costenv.sims.inedible",
    "current.intake.costenv.sims.wasted", "CF.intake.costenv.sims.wasted")

output.type.vec.summary.bystrata<-c("costenv.summary", "substitution.costenv.summary", "combined.costenvs.summary
    "delta.costenv.summary",
    #"costenv.summary.consumed", "substitution.costenv.summary.consumed",
    "combined.costenvs.summary.consumed",
    #"costenv.summary.unconsumed", "substitution.costenv.summary.unconsumed",
    "combined.costenvs.summary.unconsumed",

    "current.intake.costenv.summary", "CF.costenv.intake.summary", "current.intake.costenv
    "CF.intake.costenv.summary.consumed", "current.intake.costenv.summary.unconsumed", "CF
    "inedible.costenv.summary", "wasted.costenv.summary",
    "current.intake.costenv.summary.inedible", "CF.intake.costenv.summary.inedible",
    "current.intake.costenv.summary.wasted", "CF.intake.costenv.summary.wasted")

LB.name.vec<-c("impact_lower_bound (2.5th percentile)",
    "substitution_impact_lower_bound (2.5th percentile)",
    "combined_impact_lower_bound (2.5th percentile)",

    "delta_lower_bound (2.5th percentile)",

```

```

"combined_consumed_impact_lower_bound (2.5th percentile)",
"combined_unconsumed_impact_lower_bound (2.5th percentile)",

"current_intake_impact_lower_bound (2.5th percentile)",
"CF_intake_impact_lower_bound (2.5th percentile)",
"current_intake_consumed_impact_lower_bound (2.5th percentile)",
"CF_intake_consumed_impact_lower_bound (2.5th percentile)",
"current_intake_unconsumed_impact_lower_bound (2.5th percentile)",
"CF_intake_unconsumed_impact_lower_bound (2.5th percentile)",

"inedible_impact_lower_bound (2.5th percentile)",
"wasted_impact_lower_bound (2.5th percentile)",
"current_intake_inedible_impact_lower_bound (2.5th percentile)",
"CF_intake_inedible_impact_lower_bound (2.5th percentile)",
"current_intake_wasted_impact_lower_bound (2.5th percentile)",
"CF_intake_wasted_impact_lower_bound (2.5th percentile)"
)

```

```

median.name.vec<-c("impact_median",
                    "substitution_impact_median",
                    "combined_impact_median",

                    "delta_median",

                    "combined_consumed_impact_median",
                    "combined_unconsumed_impact_median",

                    "current_intake_impact_median",
                    "CF_intake_impact_median",
                    "current_intake_consumed_impact_median",
                    "CF_intake_consumed_impact_median",
                    "current_intake_unconsumed_impact_median",
                    "CF_intake_unconsumed_impact_median",

                    "inedible_impact_median",
                    "wasted_impact_median",
                    "current_intake_inedible_impact_median",
                    "CF_intake_inedible_impact_median",
                    "current_intake_wasted_impact_median",
                    "CF_intake_wasted_impact_median"
)

```

```

UB.name.vec<-c("impact_upper_bound (97.5th percentile)",
                "substitution_impact_upper_bound (97.5th percentile)",
                "combined_impact_upper_bound (97.5th percentile)",

                "delta_upper_bound (97.5th percentile)",

                "combined_consumed_upper_bound (97.5th percentile)",
                "combined_unconsumed_upper_bound (97.5th percentile)",

                "current_intake_impact_upper_bound (97.5th percentile)",
                "CF_intake_impact_upper_bound (97.5th percentile)",
                "current_intake_consumed_upper_bound (97.5th percentile)",
                "CF_intake_consumed_impact_upper_bound (97.5th percentile)",
                "current_intake_unconsumed_upper_bound (97.5th percentile)",
                "CF_intake_unconsumed_impact_upper_bound (97.5th percentile)",
)

```

```

        "inedible_upper_bound (97.5th percentile)",
        "wasted_unconsumed_upper_bound (97.5th percentile)",
        "current_intake_inedible_upper_bound (97.5th percentile)",
        "CF_intake_inedible_impact_upper_bound (97.5th percentile)",
        "current_intake_wasted_upper_bound (97.5th percentile)",
        "CF_intake_unconsumed_wasted_upper_bound (97.5th percentile)"
    )

mean.name.vec<-c("impact_mean",
                 "substitution_impact_mean",
                 "combined_impact_mean",

                 "delta_mean",

                 "combined_consumed_impact_mean",
                 "combined_unconsumed_impact_mean",

                 "current_intake_impact_mean",
                 "CF_intake_impact_mean",
                 "current_intake_consumed_impact_mean",
                 "CF_intake_consumed_impact_mean",
                 "current_intake_unconsumed_impact_mean",
                 "CF_intake_unconsumed_impact_mean",

                 "inedible_consumed_impact_mean",
                 "wasted_unconsumed_impact_mean",
                 "current_intake_inedible_impact_mean",
                 "CF_intake_inedible_impact_mean",
                 "current_intake_wasted_impact_mean",
                 "CF_intake_wasted_impact_mean"

)

SD.name.vec<-c("impact_SD",
               "substitution_impact_SD",
               "combined_impact_SD",

               "delta_SD",

               "combined_consumed_impact_SD",
               "combined_unconsumed_impact_SD",

               "current_intake_impact_SD",
               "CF_intake_impact_SD",
               "current_intake_consumed_impact_SD",
               "CF_intake_consumed_impact_SD",
               "current_intake_unconsumed_impact_SD",
               "CF_intake_unconsumed_impact_SD",

               "inedible_impact_SD",
               "wasted_impact_SD",
               "current_intake_inedible_impact_SD",
               "CF_intake_inedible_impact_SD",
               "current_intake_wasted_impact_SD",
               "CF_intake_wasted_impact_SD"

)

```

function to revert list structure. See link in comments for details

```
revert_list_str_4 <- function(ls) { # @Josh O'Brien
  # get sub-elements in same order
  x <- lapply(ls, `[`, names(ls[[1]]))
  # stack and reslice
  apply(do.call(rbind, x), 2, as.list)
}
#https://stackoverflow.com/questions/15263146/revert-list-structure
```

for each dietary pattern and grocery/non-grocery combo, for each output type (specified in output.type.vec.sims), combine grocery and non-grocery output and save, and, for each strata combination, save summary output files (both absolute and per-capita).

First, start the loop (through the dietary patterns). For each dietary pattern we create lists (summary.list and summary.list.percapita) to store summary stats for that dietary pattern. Note that each element of the list is itself a list containing summaries for all strata combos for a given output type.

```
for(ll in 1:length(diet.vec))
{
  diet<-diet.vec[ll]

  summary.list<-list() ##each element of the list is itself a list containing summaries for all strata combos, fo
  summary.list.percapita<-list()
```

Within the ll loop, loop through the different output types and extract sims and summary stats. Combine food at home and food away from home output and populate the lists. [see documentation for "combine.FAH.FAFH_cluster.r" for details.](#)

```
for(k in 1:length(output.type.vec.sims))
{
  summary.list[[k]]<-list()
  summary.list.percapita[[k]]<-list()

  setwd("/cluster/tufts/lasting/shared/LASTING/Code/cluster_code")
  output.type.sims<-output.type.vec.sims[k]
  output.type.summary<-output.type.vec.summary[k]
  output.type.sims.bystrata<-output.type.vec.sims.bystrata[k]
  output.type.summary.bystrata<-output.type.vec.summary.bystrata[k]
  LB.name<-LB.name.vec[k]
  median.name<-median.name.vec[k]
  UB.name<-UB.name.vec[k]

  mean.name<-mean.name.vec[k]
  SD.name<-SD.name.vec[k]

  source("combine.FAH.FAFH_cluster.r")
}
```

Still within the ll loop, revert the list structure for summary.list and summary.list.percapita so that each element corresponds to a strata combo rather than an output type.

```
summary.list.by.strata.combo<-revert_list_str_4(summary.list)
summary.list.percapita.by.strata.combo<-revert_list_str_4(summary.list.percapita)

output_location<-paste("/cluster/tufts/lasting/shared/model_development/data/outputs/cost_env/", diet, "_diet_b
```

Still within the ll loop, loop through the elements of summary.list.by.strata.combo and summary.list.percapita.by.strata.combo, and for each element (itself a list of outcomes by outcome type), merge the list of dataframes that make up that element together (and end ll loop).

```

for(mm in 1:length(names(summary.list[[1]])))
{
  setwd(paste(output_location, "/By_SubGroup", sep=""))
  list.names.intersect<-Reduce(intersect, lapply(summary.list.by.strata.combo[[mm]], colnames))
  merged<-Reduce(function(...) merge(..., by=list.names.intersect),summary.list.by.strata.combo[[mm]])
  write.csv(x=merged, file=paste("summary.output_by_", names(summary.list[[1]])[mm], ".costenv.csv", sep=""))

  setwd(paste(output_location, "/per_capita/By_SubGroup", sep=""))
  list.names.intersect<-Reduce(intersect, lapply(summary.list.percapita.by.strata.combo[[mm]], colnames))
  merged.percapita<-Reduce(function(...) merge(..., by=list.names.intersect),summary.list.percapita.by.strata.c
  write.csv(x=merged.percapita, file=paste("summary.output_by_", names(summary.list[[1]])[mm], ".costenv.csv",

}
}

```

To output how much time it took run the cost/env/FL portion of the code (find it in .txt file when you run the code).

```

end_time1<-Sys.time()

print("time it takes to do price + enironment bit: ")
print(end_time1 - start_time)

```

Now we start with **CRA portion of the code.**

List out dietary factors of interest and subset master input file by them

```

# Dietary factors vector (for CRA)
rfvec_cra <- c("dairy_tot",
              "gr_whole",
              "pf_ns",
              "pf_pm",
              "pf_redm",
              "pf_seafood",
              "veg_exc_sta",
              "ssb",
              "sodium",
              "fruit_exc_juice",
              "leg_tot")

# subset to only include dietary factors for CRA
master_input_cra <- subset(master_input, Foodgroup %in% rfvec_cra)

```

List out disease in disease vector, and create corresponding disease type vector. Type 1 = Cancer or CMD (listed as CVD in code, but it's technically CMD, Type 2 breaks down CMD into the specific diseases, but bins the rarer ones into an "other category. Type 3 merges all the strokes into one category.

```

# Diseases vector
diseases.vec <- c("CC", "MLPC", "LC", "UC", "BC", "KC",
                  "SCC", "SCNC", "LVC", "PC", "ECA", "TC",
                  "APCA", "MMC", "OC", "GC",
                  "IHD", "ISTK", "HSTK", "OSTK", "DIAB", "HHD",
                  "AA", "RHD", "ENDO", "OTH", "CM", "AFF", "PVD"
)

# Diseases type vector
# original
disease.type.vec <- c("Cancer", "Cancer", "Cancer", "Cancer", "Cancer", "Cancer",
                      "Cancer", "Cancer", "Cancer", "Cancer", "Cancer", "Cancer",
                      "Cancer", "Cancer", "Cancer", "Cancer",
                      "CVD", "CVD", "CVD", "CVD", "CVD", "CVD",
                      "CVD", "CVD", "CVD", "CVD", "CVD", "CVD", "CVD"
)

# option 1
disease.type1.vec <- disease.type.vec

# option 2
disease.type2.vec <- c("Cancer", "Cancer", "Cancer", "Cancer", "Cancer", "Cancer",
                      "Cancer", "Cancer", "Cancer", "Cancer", "Cancer", "Cancer",
                      "Cancer", "Cancer", "Cancer", "Cancer",
                      "IHD", "ISTK", "HSTK", "OSTK", "DIAB", "HHD",
                      "Other_CVD", "Other_CVD", "Other_CVD", "Other_CVD", "Other_CVD", "Other_CVD", "Other_CVD"
)

# option 3
disease.type3.vec <- c("Cancer", "Cancer", "Cancer", "Cancer", "Cancer", "Cancer",
                      "Cancer", "Cancer", "Cancer", "Cancer", "Cancer", "Cancer",
                      "Cancer", "Cancer", "Cancer", "Cancer",
                      "All_HD", "All_STK", "All_STK", "All_STK", "DIAB", "All_HD",
                      "Other_CVD", "Other_CVD", "Other_CVD", "Other_CVD", "Other_CVD", "Other_CVD", "Other_CVD"
)

# decide here which disease type option you want
# disease.type.vec <- disease.type3.vec

```

Create variable for number of diseases, and create various vectors and matrices for the “expanded” disease list, meaning each disease, but repeated three times for the three pathways: Direct effect, mediated by SBP, and mediated by BMI.

```

# Create some vectors
n.diseases <- length(diseases.vec)

pathway.vec <- c("direct", "medBMI", "medSBP")
pathway <- rep(pathway.vec, each=length(diseases.vec))

disease.table <- cbind(diseases.vec, disease.type.vec, disease.type1.vec, disease.type2.vec, disease.type3.vec)

diseases_medBMI.vec <- paste(diseases.vec, "_medBMI", sep="")
diseases_medSBP.vec <- paste(diseases.vec, "_medSBP", sep="")

diseases.vec <- c(diseases.vec, diseases_medBMI.vec, diseases_medSBP.vec)
diseases.vec.mn <- paste(diseases.vec, "mn", sep="")
diseases.vec.se <- paste(diseases.vec, "se", sep="")
num.diseases <- length(diseases.vec.mn)

disease.total.vec <- gsub(pattern="_medBMI|_medSBP", replacement="", x=diseases.vec)

# brooke edited
disease.type.total.vec <- disease.type.vec[match(disease.total.vec, disease.table)]
disease.type1.total.vec <- disease.type1.vec[match(disease.total.vec, disease.table)]
disease.type2.total.vec <- disease.type2.vec[match(disease.total.vec, disease.table)]
disease.type3.total.vec <- disease.type3.vec[match(disease.total.vec, disease.table)]

```

Set number of exposure files and mediated effects. These should not change unless you are considering adding more mediation effects (You'd have to edit the code to add those effects those, not just this line).

```

nsim3 <- 1 # number of exposure files
n.mediates.effects <- 2 # number of mediated effects

```

Read in RRs, food to BMI effects, food to SBP effects, and TMREs, and so a little data processing. (These inputs may also be in mega input file for the sake of completeness, but the actual inputs for the code are from the files listed below. A long time ago, the effects were also included in the equivalent of the "mega" file for CRA, but switched to this simpler approach)

```
##### CRA-SPECIFIC INPUTS #####

# Import relative risk (RR) data input
rrtotal = read.csv(file=paste("rr_agesexrace_", health_files_version.date, "_FINAL.csv", sep=""), header=T)

# Create age categories
rrtotal$age <- 0
rrtotal$age[rrtotal$agecat==1] <- 25
rrtotal$age[rrtotal$agecat==2] <- 35
rrtotal$age[rrtotal$agecat==3] <- 45
rrtotal$age[rrtotal$agecat==4] <- 55
rrtotal$age[rrtotal$agecat==5] <- 65
rrtotal$age[rrtotal$agecat==6] <- 75

# Import effect data inputs
Food.to.BMI.effects <- read.csv(file=paste("food_to_bmi_effects_converted_", health_files_version.date, "_FINAL.c
Food.to.SBP.effects <- read.csv(file=paste("food_to_sbp_effects_converted_", health_files_version.date, "_FINAL.c

# crate theomin (TMRED) from master_input
theomin <- master_input_cra %>%
  filter(datatype == "Total") %>%
  select(Foodgroup, TMRED_Mean_Intake, TMRED_SD_Intake, TMRED_intake_unit) %>%
  distinct() %>%
  rename(Risk_factor = Foodgroup,
         TMRED = TMRED_Mean_Intake,
         SD = TMRED_SD_Intake,
         Unit = TMRED_intake_unit)
```

Create mortality/incidence simulations.

```
## NOTE: Pop file and draws crated below

# pre-simulate mortality/incidence numbers, to be used in CRA simulations, as well for calculating reverse-engine
source(paste0(code_location, "/presimulate.observed.mort.draws_LASTING_cluster.r"))
```

Create output file path

```
# create output file path
output_location <- "/cluster/tufts/lasting/shared/model_development/data/outputs/CRA/"
# output_location <- "/Users/bmb73/Documents/GitHub/LASTING/out/CRA/"
```

Next, we set up parallel processing so we can run all five diet patterns at the same time.

```
##### RUN CRA ANALYSIS #####

# Now let's run the CRAs using parallel processing, just 5 scenarios, since we don't break down by FAH/FAFH

# Set up parallel processing
# Detect system type
get_os <- function(){
  sysinf <- Sys.info()
  if (!is.null(sysinf)){
    os <- sysinf['sysname']
    if (os == 'Darwin')
      os <- "osx"
    if (os == "windows")
      os <- "windows"
  } else { ## if we still don't know
    os <- .Platform$OS.type
    if (grepl("^darwin", R.version$os))
      os <- "osx"
    if (grepl("linux-gnu", R.version$os))
      os <- "linux"
  }
  tolower(os)
}

# Check if we are on Windows or Mac using our function.
cluster_type <- if (get_os() == "windows") {"PSOCK"} else {"FORK"}

no_cores <- detectCores() - 1
s = 1 # initialize iteration variable before forking

cl <- makeCluster(no_cores, type=cluster_type) # Make the cluster
# clusterEvalQ(cl)
registerDoParallel(cl)
```

Start foreach loop. Inside the loop is the code to run the cra. Recall that diet.vec is a vector listing all dietary patterns of interest. The foreach loop allows us to run the code (in this case, the CRA code) within the loop in parallel for each element in diet.vec.

```
# This is the for loop that runs the CRA analysis for all dietary patterns
foreach(ss=1:length(diet.vec), .verbose=TRUE) %dopar% {
```

Read in master input file, subset by dietary pattern of interest, and do some data processing. Also set seed within the for loop with the hope that any duplicated simulations will be consistent among dietary patterns.

```

set.seed(359)

# update file path
setwd(file_location)

# Read CSV file of exposure (name of master input file to be finalized, potentially changes)
expos = read.csv(file=paste("mega_costenv_structure_", version.date, "_FINAL.csv", sep=""), header=T)
expos$diet_pattern[expos$diet_pattern=="Vegetarian"] <- "Veg"
expos <- subset(expos, Foodgroup %in% rfvec_cra)

expos <- subset(expos, diet_pattern == diet.vec[ss] & datatype=="Total")
expos <- expos[order(expos$Foodgroup),]

as.data.frame(table(expos$Foodgroup, useNA = "always"))

# Fix a bunch of variable names
expos$riskfactor <- expos$Foodgroup ##renaming it in case name of variable ever changes in datafile.
expos$diet <- expos$Foodgroup ##renaming it in case name of variable ever changes in datafile.
expos$sd <- expos$sigma_u_wgt ##change this line if you want to use total sd or unweighted sd or something else
expos$sd.alt <- expos$sigma_u_wgt.alt ##change this line if you want to use total sd or unweighted sd or someth
expos$mean <- expos$Mean_Intake
expos$se <- expos$SE_Intake
expos$subgroup <- expos$subgroup_id
expos$CF_mean_intake <- expos$CF_Mean_Intake
expos$CF_se_intake <- expos$CF_SE_Intake
expos$age <- expos$Age
expos$agecat <- expos$Age
#expos$race<-expos$Race
expos$race <- 0
expos$race[expos$Race_label=="NHW"] <- 1
expos$race[expos$Race_label=="NHB"] <- 2
expos$race[expos$Race_label=="HIS"] <- 3
expos$race[expos$Race_label=="OTH"] <- 4
expos$female <- 0
expos[expos$Sex==1,]$female <- 1

# Create # of strata variable
n.strata <- expos %>% select(subgroup_id) %>% distinct() %>% nrow()

# Assign age
expos$age <- 0
expos$age[expos$agecat==1] <- 25
expos$age[expos$agecat==2] <- 35
expos$age[expos$agecat==3] <- 45
expos$age[expos$agecat==4] <- 55
expos$age[expos$agecat==5] <- 65
expos$age[expos$agecat==6] <- 75

expos$age.mid<-0
expos$age.mid[expos$agecat==1] <- median(c(25,35))
expos$age.mid[expos$agecat==2] <- median(c(35,45))
expos$age.mid[expos$agecat==3] <- median(c(45,55))
expos$age.mid[expos$agecat==4] <- median(c(55,65))
expos$age.mid[expos$agecat==5] <- median(c(65,75))
expos$age.mid[expos$agecat==6] <- median(c(75,100))
.GlobalEnv$expos <- expos

```

Add variable to specify number of risk factors, variable to specify length of exposure dataset for one risk factor (in our case, that's the number of subgroups) and variable specifying name of dietary of pattern. Note some of these variables needed to be added to the global environment for the code to work with parallel processing. Note also that expos data frame also was added to global variable in previous section.

```
.GlobalEnv$expos <- expos

expos.rf=length(unique(expos$riskfactor)) # number of risk factors in the file (e.g., 1 if there is only one ri

nsim2 <- length(expos$mean)/expos.rf # Length of exposure dataset for one risk factor (i.e. rows in exposure da

# add var to global environment
.GlobalEnv$nsim2 <- nsim2

# change working directory to cluster code folder
setwd(code_location)

# set diet pattern
diet_pattern <- diet.vec[ss]

# add var to global environment
.GlobalEnv$diet_pattern <- diet_pattern

# save data so I don't have to run everything again when debugging
# comment this out when running on Tufts cluster!
# save.image(paste0(code_location, "/debug/BeforeRun.RData"))
```

Now that we have read in everything we need, next is running the model. Part 1 actually calculates the PIFs for all diet-disease pairs of interest for all subgroups, x times depending on what you set nsim to be. The entire simulation results will be saved. Part 2 creates summary results, for all possible subgroup combinations. Part 3 calculates joint PIFs (for all subgroups).

```

# Part 1
source("Complete.PIF.Analysis.PartOne.lasting.cluster.r") # Actually runs the simulations and calculates PIF for

# save data so I don't have to run everything again when debugging
# comment this out when running on Tufts cluster!
# save.image(paste0(code_location, "/debug/Part1.RData"))

# Load data
# Load(paste0(code_location, "/debug/Part1.RData"))

# Part 2
source("Complete.PIF.Analysis.partTwo.cluster.r")

# save data so I don't have to run everything again when debugging
# comment this out when running on Tufts cluster!
# save.image(paste0(code_location, "/debug/Part2.RData"))

# Load data
# Load(paste0(code_location, "/debug/Part2.RData"))

# change working directory to cluster code folder
setwd(code_location)

# Part 3
# source("Complete.PIF.Analysis.partThree.cluster.r") # Calculates Joint PIFs for each strata and each disease
source(paste0(code_location, "/debug/Part3_debug_041824.R")) #temp

# save data so I don't have to run everything again when debugging
# comment this out when running on Tufts cluster!
# save.image(paste0(code_location, "/debug/Part3.RData"))

```

End of foreach loop

```
}

```

We're done. End the timer, stop the set up to run simultaneously, and print how much time it took to run the code.

```

end_time2 <- Sys.time()

stopCluster(cl)

print("time it takes to do CRA bit: ")

# print(end_time2 - end_time1)
print(end_time2 - start_time)

##### END OF CODE #####

```